

Crivo

Desafios de pixel art

Copyright (©) 2026 Erik Pimenta

Versão 2.4.0 · Novembro de 2026 · Todos os direitos reservados

Versão 2.4.0 · Novembro de 2026

Novidades da versão 2.4.0

A versão 2.4.0 traz o **Estúdio de jogo** (menu **Jogo**, `F9`): os desenhos do Crivo viram sprites e tiles de um jogo 2D, com cenas, atores, regras, eventos e código em Crivo Script, C++ e Lua.

Este caderno ganhou seis **desafios de jogo**, numerados de 07 a 12, a partir da página 5. Cada um tem um modelo do resultado e uma lista de passos para marcar.

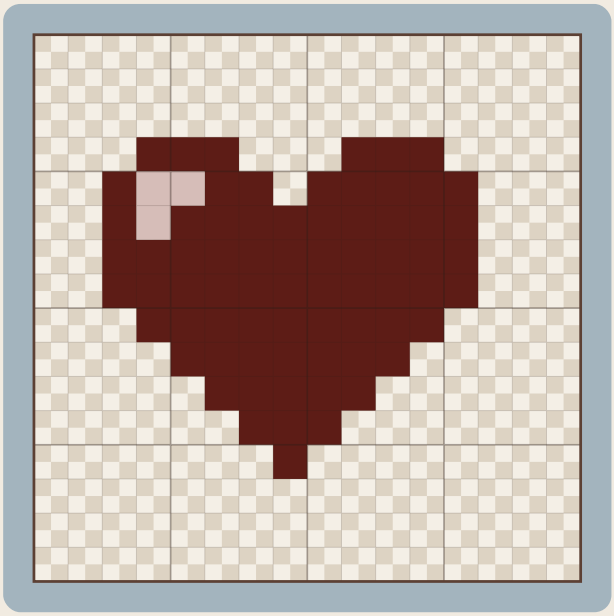
Desafio	Nível	O que pratica
07 · Primeira fase	Nível 1	Plataforma
08 · Inimigo com eventos	Nível 2	Eventos
09 · Placar e regras	Nível 2	Regras
10 · Função em Lua	Nível 3	Código
11 · Função em C++	Nível 3	Código
12 · Terminal e depuração	Nível 3	Testes

> Use o jogo de exemplo **Gotinha e as moedas** como referência: ele abre na primeira vez que você entra no estúdio e também está em `examples/gotinha.crivojogo`.

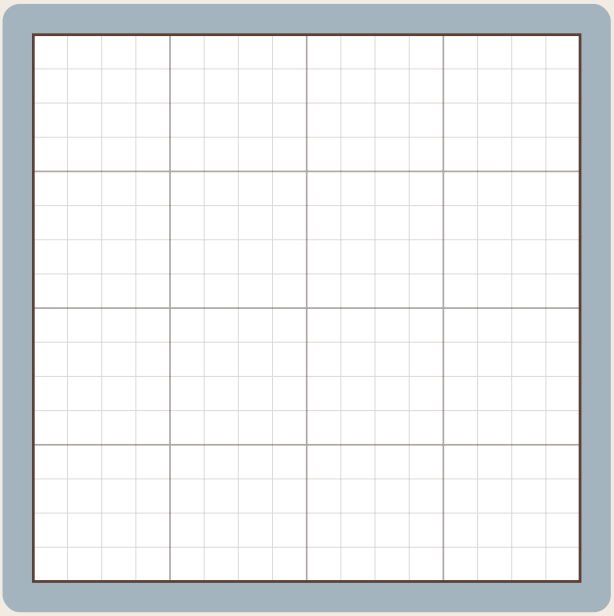
01 Coração

Nível 1 · 16 × 16

MODELO



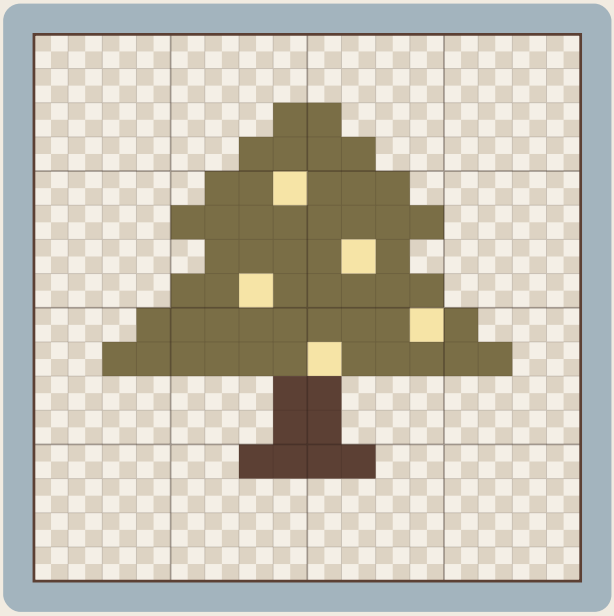
SUA VEZ



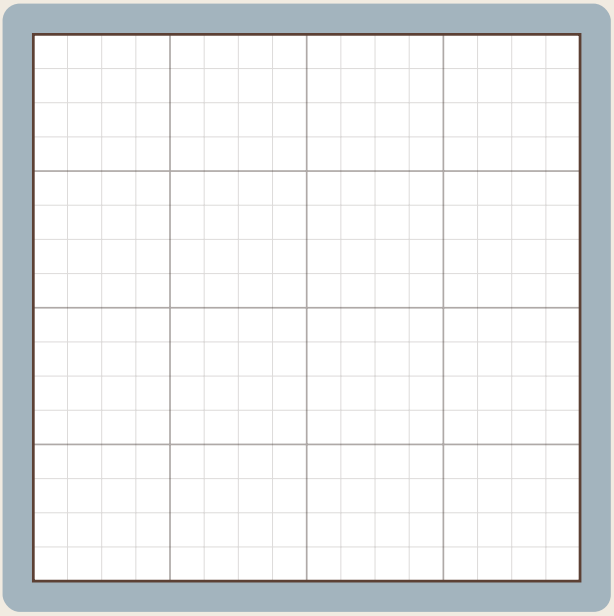
02 Árvore

Nível 1 · 16 × 16

MODELO



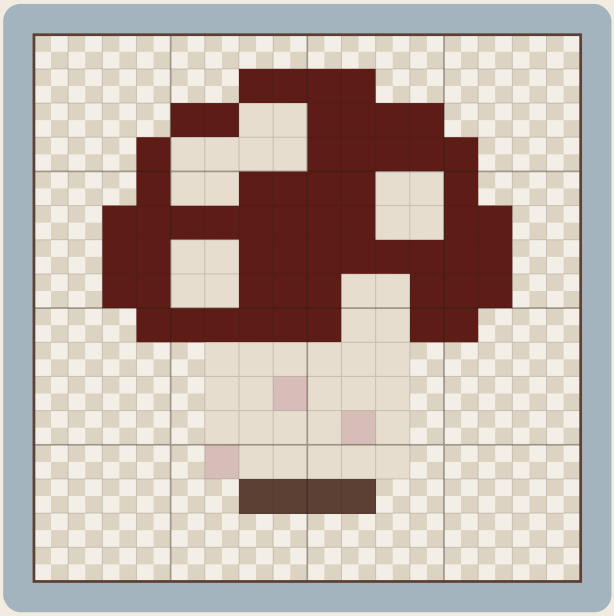
SUA VEZ



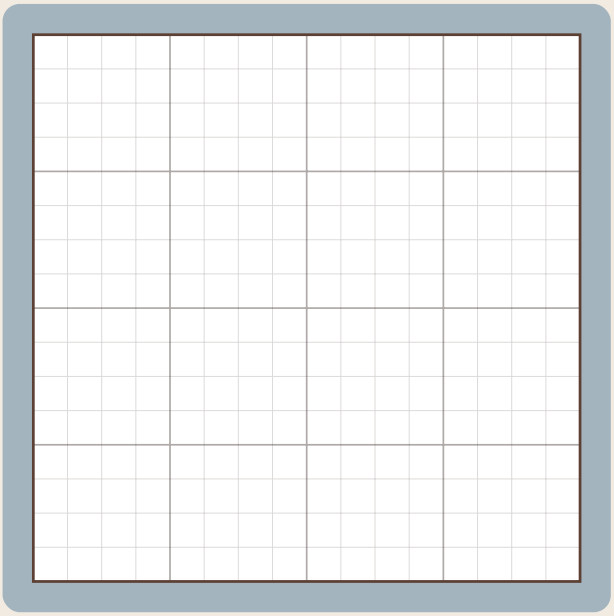
03 Cogumelo

Nível 2 · 16 × 16

MODELO



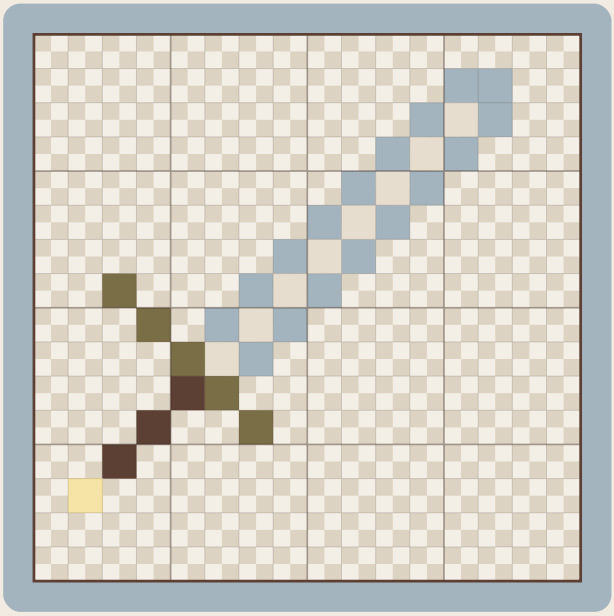
SUA VEZ



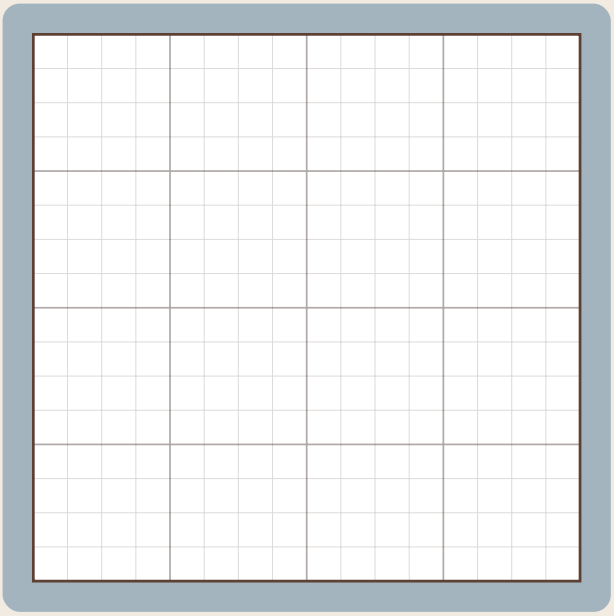
04 Espada

Nível 2 · 16 × 16

MODELO



SUA VEZ



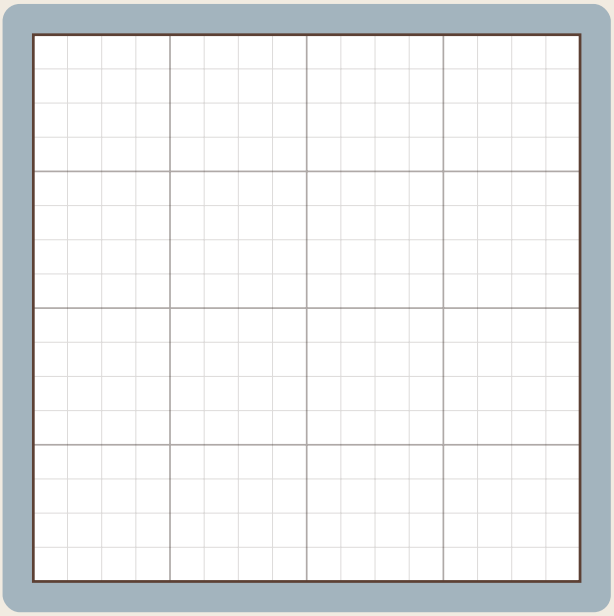
05 Gato

Nível 3 · 16 × 16

MODELO



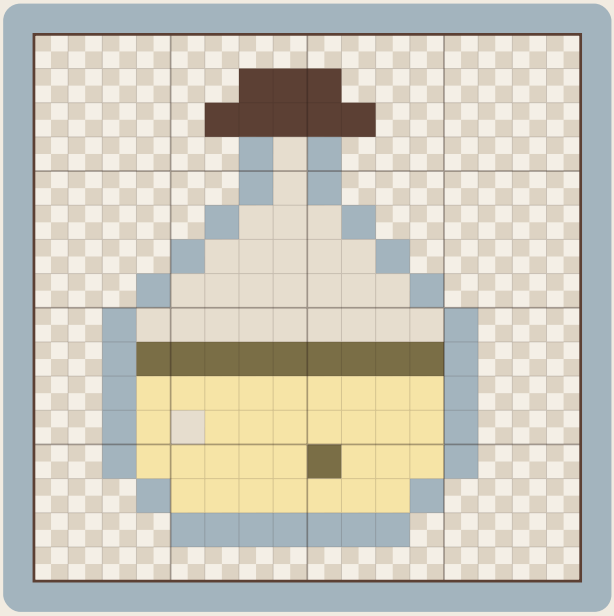
SUA VEZ



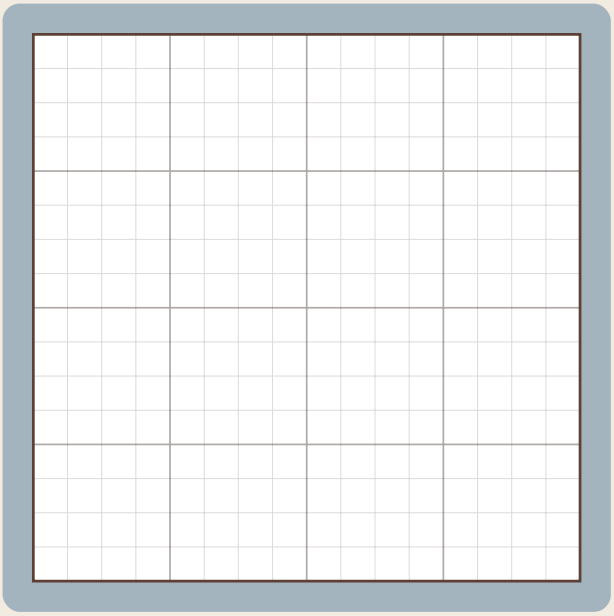
06 Poção

Nível 3 · 16 × 16

MODELO



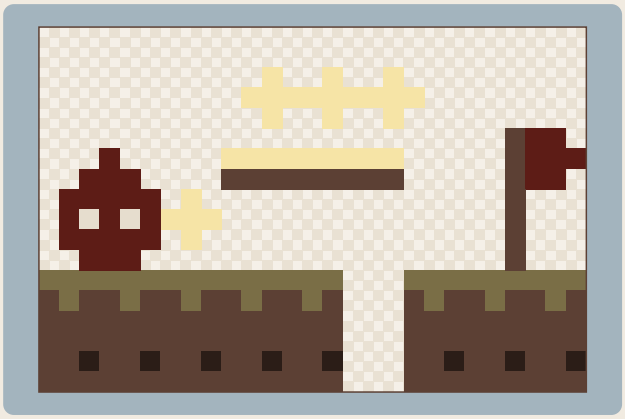
SUA VEZ



07 Primeira fase

Nível 1 · Plataforma

MODELO



Monte uma fase curta: o jogador sai da esquerda, pega as moedas, pula o buraco e chega à bandeira.

SUA VEZ

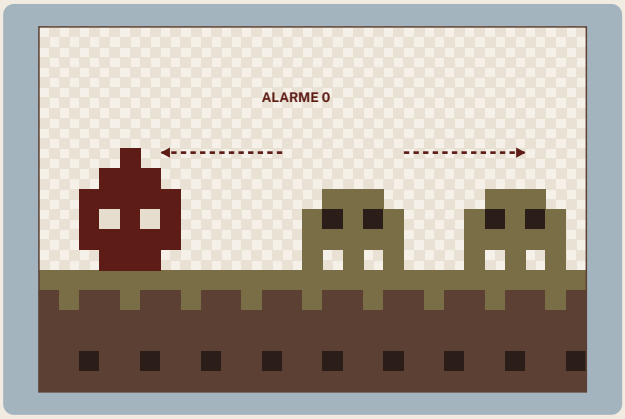
- [ ] Abra o estúdio (`F9`) e use **Arquivo ▾ → Novo jogo em branco**.
- [ ] Na aba **Tiles**, clique em **Novo tileset** e marque o chão como **sólido** e uma tábua como **plataforma**.
- [ ] Crie os atores com **Novo ator**: o herói como **Jogador (plataforma)**, a moeda como **Item coletável** e a bandeira como **Saída**.
- [ ] Na aba **Cenas**, pinte o chão com **Pintar tile** (`B`) e deixe um buraco de um tile.
- [ ] Coloque os atores com **Colocar ator** (`A`): o herói à esquerda, cinco moedas e a bandeira no fim.
- [ ] Teste com `F5` : pule o buraco e chegue à saída.

Dica: ligue **Colisão** na barra da cena para ver quais tiles seguram o jogador.

08 Inimigo com eventos

Nível 2 · Eventos

MODELO



Crie um inimigo que vira de lado sozinho de tempos em tempos e machuca o jogador quando encosta.

SUA VEZ

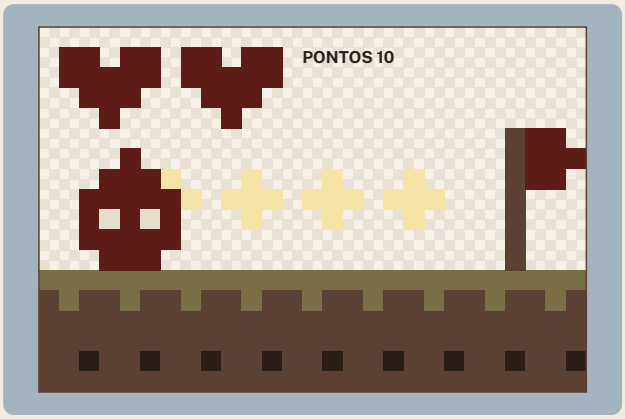
- [ ] Crie o ator **Gosma** com o comportamento **Objeto livre (só eventos)**, com gravidade e colisão com os tiles.
- [ ] Evento **Criar**: ação **Mover numa direção** com 0 graus (direita) e **Ligar alarme** 0 em 2 segundos.
- [ ] Evento **Alarme 0**: **Inverter movimento** e ligar o alarme 0 de novo em 2 segundos.
- [ ] Evento **Colisão com** o herói: **Dar dano** em **o outro**.
- [ ] Coloque três gosmas na cena e teste com `F5` .

Dica: selecione uma gosma na cena e aperte `F` para espelhá-la.

09 Placar e regras

Nível 2 · Regras

MODELO



Mostre pontos e vidas no placar, dê uma vida extra com 10 pontos e vença quando acabarem as moedas.

SUA VEZ

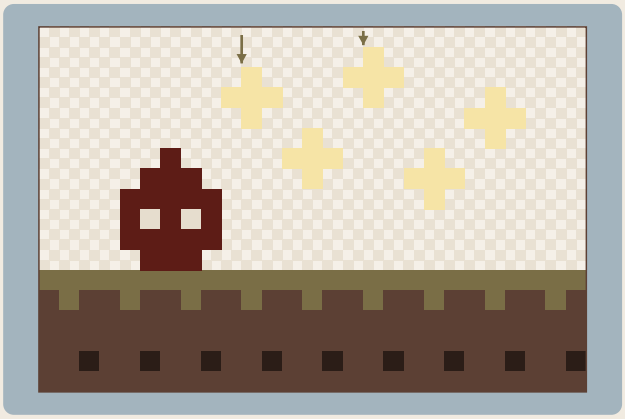
- [ ] Na aba **Regras**, crie as variáveis **pontos** e **vidas** e marque **no placar**.
- [ ] Na aba **Jogo**, escolha essas variáveis como pontos e vidas.
- [ ] Regra: **ao começar a cena**, mostrar a mensagem “Pegue as moedas!” por 3 segundos.
- [ ] Regra: quando a variável **pontos** chegar a 10, somar 1 em **vidas** e tocar o som **Power-up**.
- [ ] Regra: quando **nenhum ator** Moeda sobrar, **vencer**.
- [ ] Na aba **Sons**, crie o Power-up a partir do modelo pronto e experimente o botão **Aleatório**.

Dica: mensagens aceitam variáveis entre chaves, como “Você tem {pontos} pontos”.

10 Função em Lua

Nível 3 · Código

MODELO



Escreva em Lua uma função que faz chover moedas perto do jogador e chame-a de um evento.

SUA VEZ

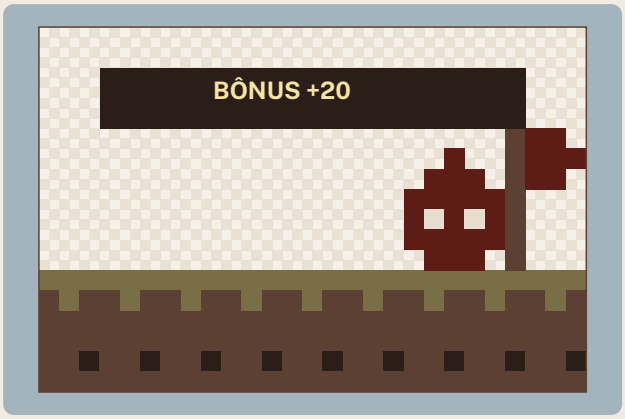
- [ ] Na aba **Código**, crie um script com a linguagem **Lua**.
- [ ] Escreva a função abaixo:
- [ ] No herói, crie um evento **Tecla apertada** com a ação **Executar código**: `chuva(5)` .
- [ ] Teste com `F5` , aperte a tecla e confira no terminal (`F4`) com `instance_number("Moeda")` .

```
function chuva(n)
  for i = 1, n do
    instance_create("Moeda", x + i * 16, y - 32)
  end
end
```

Dica: em Lua, tabelas começam no índice 1, # dá o tamanho e .. junta textos.

11 Função em C++ Nível 3 · Código

MODELO



Calcule um bônus em C++ e some o resultado aos pontos quando o jogador chega à saída.

SUA VEZ

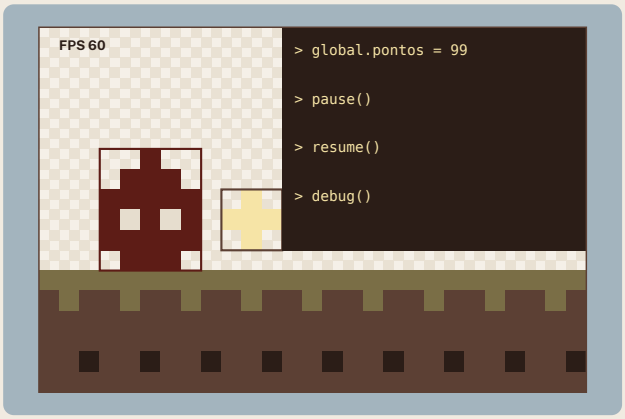
- [ ] Na aba **Código**, crie um script com a linguagem **C++ adaptado**.
- [ ] Escreva a função abaixo:
- [ ] Crie a regra **quando o jogador chegar à saída** com **Executar código** em Crivo Script: `global.pontos += bonus(global.vidas)` .
- [ ] Teste com 2 vidas: 2 × 10 / 3 dá 6, porque `int` corta a parte decimal.

```
int bonus(int vidas) {
  int extra = vidas * 10 / 3;
  std::cout << "bônus " << extra << std::endl;
  return extra;
}
```

Dica: #include e using namespace são ignorados; pode deixá-los no código.

12 Terminal e depuração Nível 3 · Testes

MODELO



Teste a fase pelo terminal e pela depuração, sem precisar jogar tudo de novo.

SUA VEZ

- [ ] No terminal da aba **Código**, rode `instance_number("Moeda")` e anote o número.
- [ ] Rode `step(60)` para avançar 1 segundo e `reset()` para voltar ao início.
- [ ] Teste com `F5` , abra o **Terminal** (`F4`) e digite `global.pontos = 99` .
- [ ] Crie uma moeda ao lado do jogador: `instance_create("Moeda", x + 16, y)` .
- [ ] Use `pause()` e `resume()` ; ligue a **Depuração** (`F3`) e veja as caixas de colisão e o FPS.

Dica: digite `ajuda` para listar as funções; as setas trazem os comandos anteriores.